

Лабораторная работа 1

Цель: исследование моделей классификации изображений: бейзлайн (torchvision CNN и ViT), улучшение бейзлайна, собственная архитектура и сравнения.

Данные: Bone Break Classification (Kaggle), изображения в репозитории под `data/Bone Break Classification/`.

Метрики: accuracy; precision, recall, F1 (macro и weighted) на валидации; финальная оценка на отложенном test без участия в подборе гиперпараметров.

Воспроизводимость и устройство

Константа `SEED` фиксирует сплиты и инициализацию. `DEVICE_MODE` задаёт вычислитель: `auto` (CUDA, иначе MPS, иначе CPU), либо явно `mps`, `cuda`, `cpu`. Число эпох по моделям задаётся словарём `EPOCHS_BY_DEVICE` (ключи `cpu` / `mps` / `cuda` и теги моделей); в обучении используется `epochs_for("...")`.

Hold-out каталоги `*/Test` не используются при обучении и выборе гипотез; валидация — из доли `Train` (стратификация по классу).

```
In [1]: import os
import random
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
)
from torchvision import transforms, models
from torchvision.models import ResNet18_Weights, ViT_B_16_Weights
from tqdm.auto import tqdm

SEED = 42
DEVICE_MODE = "auto"
```

```

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

def resolve_device(mode: str) -> torch.device:
    mode = mode.lower().strip()
    if mode == "cuda":
        return torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if mode == "mps":
        if hasattr(torch.backends, "mps") and torch.backends.mps.is_avail
            return torch.device("mps")
        return torch.device("cpu")
    if mode == "cpu":
        return torch.device("cpu")
    if torch.cuda.is_available():
        return torch.device("cuda")
    if hasattr(torch.backends, "mps") and torch.backends.mps.is_available
        return torch.device("mps")
    return torch.device("cpu")

device = resolve_device(DEVICE_MODE)
if device.type == "cuda":
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

g = torch.Generator()
g.manual_seed(SEED)
print("device:", device, "| DEVICE_MODE:", DEVICE_MODE)

weight_decay = 0.01

EPOCHS_BY_DEVICE = {
    "cpu": {
        "ResNet18": 4,
        "ViT-B/16": 4,
        "ResNet18+": 4,
        "ViT-B/16+": 4,
        "SmallCNN": 5,
        "SmallCNN+": 5,
    },
    "mps": {
        "ResNet18": 50,
        "ViT-B/16": 50,
        "ResNet18+": 50,
        "ViT-B/16+": 50,
        "SmallCNN": 50,
        "SmallCNN+": 50,
    },
    "cuda": {
        "ResNet18": 50,
        "ViT-B/16": 50,
        "ResNet18+": 50,
        "ViT-B/16+": 50,
        "SmallCNN": 50,
        "SmallCNN+": 50,
    },
}

```

```

}

def epochs_for(model_tag: str) -> int:
    table = EPOCHS_BY_DEVICE.get(device.type) or EPOCHS_BY_DEVICE["cpu"]
    if model_tag not in table:
        raise KeyError(f"нет ключа {model_tag!r} в EPOCHS_BY_DEVICE[{device.type}]")
    return int(table[model_tag])

print(
    "epochs_by_model:",
    {k: epochs_for(k) for k in EPOCHS_BY_DEVICE.get(device.type, EPOCHS_BY_DEVICE["cpu"])}
)

```

```

device: mps | DEVICE_MODE: auto
epochs_by_model: {'ResNet18': 50, 'ViT-B/16': 50, 'ResNet18+': 50, 'ViT-B/16+': 50, 'SmallCNN': 50, 'SmallCNN+': 50}

```

1. Данные: индексация и сплиты

Корень `DATA_ROOT`: вложенная папка экспорта. Для каждого класса читаются пути из `Train` и `Test`. Из объединённого `Train` стратифицированно выделяется `val` (`val_ratio`). `Dataset` принимает список пар (путь, метка) и `transform`.

```

In [2]: _cwd = Path.cwd().resolve()
if (_cwd / "data" / "Bone Break Classification").is_dir():
    ROOT = _cwd
elif (_cwd / "lab_1" / "data" / "Bone Break Classification").is_dir():
    ROOT = _cwd / "lab_1"
else:
    raise FileNotFoundError(
        "Ожидается запуск из lab_1 или из корня репозитория (cyber_sys),
    )

DATA_ROOT = ROOT / "data" / "Bone Break Classification" / "Bone Break Cla
# DATA_ROOT = ROOT / "data" / "Bone Break Classification"
assert DATA_ROOT.is_dir(), f"Нет каталога: {DATA_ROOT}"

IMG_EXT = {".jpg", ".jpeg", ".png", ".JPG", ".JPEG", ".PNG"}
val_ratio = 0.2

def collect_split(class_name: str, split: str):
    d = DATA_ROOT / class_name / split
    if not d.is_dir():
        return []
    out = []
    for p in d.rglob("*"):
        if p.suffix in IMG_EXT:
            out.append(p)
    return sorted(out)

class_names = sorted(
    [p.name for p in DATA_ROOT.iterdir() if p.is_dir() and not p.name.ista
)

```

```

class_to_idx = {c: i for i, c in enumerate(class_names)}
num_classes = len(class_names)

records_train = []
records_test = []
for c in class_names:
    yi = class_to_idx[c]
    for p in collect_split(c, "Train"):
        records_train.append((str(p), yi))
    for p in collect_split(c, "Test"):
        records_test.append((str(p), yi))

paths_tr = [a for a, _ in records_train]
labels_tr = [b for _, b in records_train]
paths_fit, paths_val, y_fit, y_val = train_test_split(
    paths_tr,
    labels_tr,
    test_size=val_ratio,
    random_state=SEED,
    stratify=labels_tr,
)
train_fit = list(zip(paths_fit, y_fit))
val_records = list(zip(paths_val, y_val))

print("classes:", num_classes, class_names)
print("train_fit:", len(train_fit), "val:", len(val_records), "test:", le

class ImageListDataset(Dataset):
    def __init__(self, items, transform=None):
        self.items = items
        self.transform = transform

    def __len__(self):
        return len(self.items)

    def __getitem__(self, i):
        path, y = self.items[i]
        img = Image.open(path).convert("RGB")
        if self.transform is not None:
            img = self.transform(img)
        return img, y

```

```

classes: 10 ['Avulsion fracture', 'Comminuted fracture', 'Fracture Dislocation', 'Greenstick fracture', 'Hairline Fracture', 'Impacted fracture', 'Longitudinal fracture', 'Oblique fracture', 'Pathological fracture', 'Spiral Fracture']
train_fit: 791 val: 198 test: 140

```

1a. Обоснование выбора датасета

Мультиклассовая классификация рентген/иллюстраций по типу перелома применима к вспомогательной диагностике и обучению специалистов: автоматическая предварительная категоризация снимков снижает нагрузку на радиолога при массовом скрининге. Набор размечен по клинически осмысленным классам (вольтман, спиральный и др.), объём ограничен, что отражает реалистичный сценарий малых медицинских выборок.

Уникальность набора в рамках группы необходимо согласовать с преподавателем; публичный источник указан в README.

1b. Метрики

Ассурасу показывает долю верных среди всех классов. При разном числе объектов по классам (дисбаланс) ориентир дополняют **macro** precision/recall/F1 (равный вес классов) и **weighted** F1 (вес пропорционален поддержке в выборке). Для медицинской задачи важны и пропуск редкого класса, и ложные тревоги; отчёт включает `classification_report` и матрицу ошибок на валидации и на финальном test.

Test-разметка Roboflow мала по классам; итоговые цифры на test интерпретировать вместе с val.

EDA

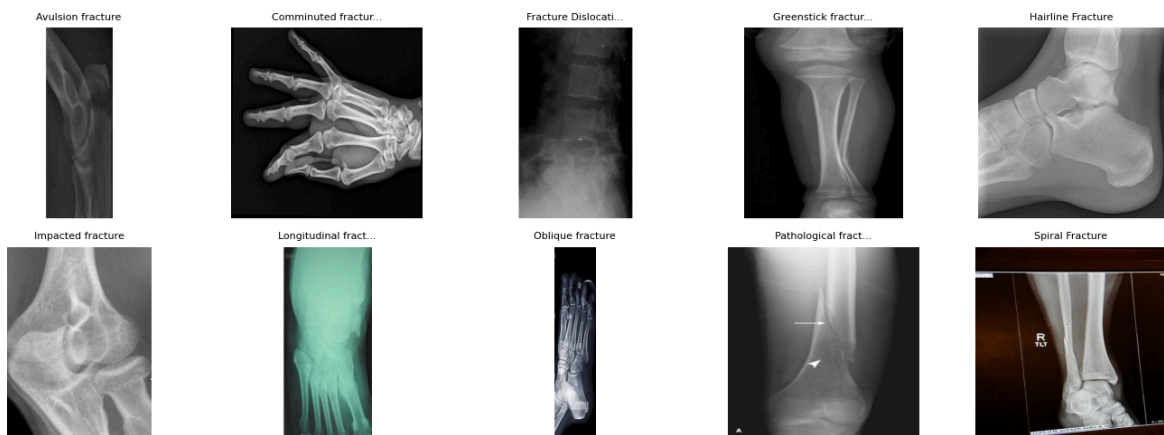
Сводка по числу файлов в `Train / Test` и примеры изображений по одному на класс (из `train_fit`).

```
In [3]: counts = {c: {"Train": len(collect_split(c, "Train")), "Test": len(collect_split(c, "Test"))} for c in class_names}
df_counts = pd.DataFrame(counts).T
display(df_counts)
print("total train files:", len(paths_tr), "total test files:", len(paths_te))

fig, axes = plt.subplots(2, 5, figsize=(14, 5))
axes = axes.flatten()
for ax, c in zip(axes, class_names):
    pts = collect_split(c, "Train")
    if not pts:
        ax.axis("off")
        continue
    im = Image.open(pts[0]).convert("RGB")
    ax.imshow(im)
    ax.set_title(c[:18] + ("..." if len(c) > 18 else ""), fontsize=8)
    ax.axis("off")
plt.tight_layout()
plt.show()
```

	Train	Test
Avulsion fracture	109	14
Comminuted fracture	134	14
Fracture Dislocation	137	19
Greenstick fracture	106	16
Hairline Fracture	101	10
Impacted fracture	75	9
Longitudinal fracture	68	12
Oblique fracture	69	16
Pathological fracture	116	18
Spiral Fracture	74	12

total train files: 989 total test files: 140



Трансформы и загрузчики (бейзлайн)

Для ResNet18 и ViT используются стандартные препроцессы весов IMAGENET1K. Обучающий набор: resize/crop + лёгкая аугментация; val/test: только детерминированная нормализация под те же веса.

```
In [4]: weights_res = ResNet18_Weights.IMAGENET1K_V1
weights_vit = ViT_B_16_Weights.IMAGENET1K_V1
pre_tf_res = weights_res.transforms()
pre_tf_vit = weights_vit.transforms()

train_tf_res = transforms.Compose(
    [
        transforms.RandomResizedCrop(pre_tf_res.crop_size, antialias=True),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_res.mean, std=pre_tf_res.std),
    ]
)
val_tf_res = transforms.Compose(
    [
        transforms.Resize(pre_tf_res.resize_size, antialias=True),
        transforms.CenterCrop(pre_tf_res.crop_size),
```

```

        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_res.mean, std=pre_tf_res.std),
    ]
)

train_tf_vit = transforms.Compose(
    [
        transforms.RandomResizedCrop(pre_tf_vit.crop_size, antialias=True),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_vit.mean, std=pre_tf_vit.std),
    ]
)

val_tf_vit = transforms.Compose(
    [
        transforms.Resize(pre_tf_vit.resize_size, antialias=True),
        transforms.CenterCrop(pre_tf_vit.crop_size),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_vit.mean, std=pre_tf_vit.std),
    ]
)

batch_size = 16
train_ds_res = ImageListDataset(train_fit, train_tf_res)
val_ds_res = ImageListDataset(val_records, val_tf_res)
test_ds_res = ImageListDataset(records_test, val_tf_res)

train_ds_vit = ImageListDataset(train_fit, train_tf_vit)
val_ds_vit = ImageListDataset(val_records, val_tf_vit)
test_ds_vit = ImageListDataset(records_test, val_tf_vit)

loader_train_res = DataLoader(
    train_ds_res,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)

loader_val_res = DataLoader(val_ds_res, batch_size=batch_size, shuffle=False)
loader_test_res = DataLoader(test_ds_res, batch_size=batch_size, shuffle=False)

loader_train_vit = DataLoader(
    train_ds_vit,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)

loader_val_vit = DataLoader(val_ds_vit, batch_size=batch_size, shuffle=False)
loader_test_vit = DataLoader(test_ds_vit, batch_size=batch_size, shuffle=False)

```

2. Бейзлайн: обучение и метрики

Две модели из `torchvision` : ResNet18 и ViT-B/16, предобученные на ImageNet-1K, замена классификационной головы на `num_classes` .

Оптимизатор AdamW, кросс-энтропия. Оценка на `val` по эпохам; после обучения однократная оценка на `test` (не использовалась при настройке).

```
In [5]: @torch.no_grad()
def evaluate_cls(model, loader, device):
    model.eval()
    ys, ps = [], []
    for x, y in loader:
        x = x.to(device, non_blocking=True)
        logits = model(x)
        pred = logits.argmax(dim=1).cpu().numpy()
        ys.append(y.numpy())
        ps.append(pred)
    y_true = np.concatenate(ys)
    y_pred = np.concatenate(ps)
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision_macro": precision_score(y_true, y_pred, average="macro", zero_division=0),
        "recall_macro": recall_score(y_true, y_pred, average="macro", zero_division=0),
        "f1_macro": f1_score(y_true, y_pred, average="macro", zero_division=0),
        "f1_weighted": f1_score(y_true, y_pred, average="weighted", zero_division=0),
        "y_true": y_true,
        "y_pred": y_pred,
    }

def train_model(model, loader_train, loader_val, epochs, lr, device, tag="train"):
    model = model.to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
    crit = nn.CrossEntropyLoss()
    hist = []
    for ep in range(1, epochs + 1):
        model.train()
        running = 0.0
        n = 0
        for x, y in tqdm(loader_train, desc=f"{tag} ep{ep}/{epochs}", leave=False):
            x = x.to(device, non_blocking=True)
            y = y.to(device, non_blocking=True)
            opt.zero_grad(set_to_none=True)
            logits = model(x)
            loss = crit(logits, y)
            loss.backward()
            opt.step()
            running += loss.item() * x.size(0)
            n += x.size(0)
        tr_loss = running / max(n, 1)
        m = evaluate_cls(model, loader_val, device)
        hist.append({"epoch": ep, "train_loss": tr_loss, **{k: m[k] for k in m}})
        print(f"{tag} ep{ep} loss={tr_loss:.4f} val_acc={m['accuracy']:.4f}")
    return hist

LR_RES = 1e-4
LR_VIT = 5e-5

model_res = models.resnet18(weights=weights_res)
model_res.fc = nn.Linear(model_res.fc.in_features, num_classes)
```



```

hist_res = train_model(
    model_res, loader_train_res, loader_val_res, epochs_for("ResNet18"),
)

model_vit = models.vit_b_16(weights=weights_vit)
in_f = model_vit.heads.head.in_features
model_vit.heads.head = nn.Linear(in_f, num_classes)
hist_vit = train_model(
    model_vit, loader_train_vit, loader_val_vit, epochs_for("ViT-B/16"),
)

```

```

ResNet18 ep1/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep1 loss=2.2715 val_acc=0.2677 f1_macro=0.2298
ResNet18 ep2/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep2 loss=1.9441 val_acc=0.3030 f1_macro=0.2582
ResNet18 ep3/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep3 loss=1.8033 val_acc=0.3990 f1_macro=0.3466
ResNet18 ep4/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep4 loss=1.7183 val_acc=0.3838 f1_macro=0.3351
ResNet18 ep5/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep5 loss=1.6193 val_acc=0.3485 f1_macro=0.2972
ResNet18 ep6/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep6 loss=1.5056 val_acc=0.3889 f1_macro=0.3724
ResNet18 ep7/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep7 loss=1.4371 val_acc=0.3939 f1_macro=0.3662
ResNet18 ep8/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep8 loss=1.3674 val_acc=0.4444 f1_macro=0.4207
ResNet18 ep9/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18 ep9 loss=1.3404 val_acc=0.4444 f1_macro=0.4024
ResNet18 ep10/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep10 loss=1.2874 val_acc=0.3990 f1_macro=0.3817
ResNet18 ep11/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep11 loss=1.1632 val_acc=0.4646 f1_macro=0.4345
ResNet18 ep12/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep12 loss=1.0795 val_acc=0.4545 f1_macro=0.4336
ResNet18 ep13/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep13 loss=1.0622 val_acc=0.4545 f1_macro=0.4249
ResNet18 ep14/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep14 loss=0.9492 val_acc=0.4697 f1_macro=0.4530
ResNet18 ep15/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep15 loss=0.9441 val_acc=0.4646 f1_macro=0.4272
ResNet18 ep16/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep16 loss=0.8787 val_acc=0.4798 f1_macro=0.4422
ResNet18 ep17/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep17 loss=0.8768 val_acc=0.4747 f1_macro=0.4592
ResNet18 ep18/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep18 loss=0.9125 val_acc=0.4848 f1_macro=0.4707
ResNet18 ep19/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep19 loss=0.8576 val_acc=0.5253 f1_macro=0.4963
ResNet18 ep20/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep20 loss=0.7368 val_acc=0.4444 f1_macro=0.4189
ResNet18 ep21/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep21 loss=0.8567 val_acc=0.5000 f1_macro=0.4723
ResNet18 ep22/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep22 loss=0.8586 val_acc=0.4848 f1_macro=0.4572
ResNet18 ep23/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep23 loss=0.8016 val_acc=0.4495 f1_macro=0.4261
ResNet18 ep24/50: 0%|         | 0/50 [00:00<?, ?it/s]
ResNet18 ep24 loss=0.7052 val_acc=0.5051 f1_macro=0.4826
ResNet18 ep25/50: 0%|         | 0/50 [00:00<?, ?it/s]

```

```
ResNet18 ep25 loss=0.6756 val_acc=0.5000 f1_macro=0.4586
ResNet18 ep26/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep26 loss=0.7019 val_acc=0.4848 f1_macro=0.4587
ResNet18 ep27/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep27 loss=0.6895 val_acc=0.4949 f1_macro=0.4743
ResNet18 ep28/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep28 loss=0.7066 val_acc=0.4848 f1_macro=0.4680
ResNet18 ep29/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep29 loss=0.6298 val_acc=0.5101 f1_macro=0.4823
ResNet18 ep30/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep30 loss=0.6491 val_acc=0.4495 f1_macro=0.4305
ResNet18 ep31/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep31 loss=0.6960 val_acc=0.5101 f1_macro=0.4904
ResNet18 ep32/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep32 loss=0.6569 val_acc=0.4949 f1_macro=0.4822
ResNet18 ep33/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep33 loss=0.5935 val_acc=0.4899 f1_macro=0.4627
ResNet18 ep34/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep34 loss=0.5885 val_acc=0.5202 f1_macro=0.4991
ResNet18 ep35/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep35 loss=0.5908 val_acc=0.5606 f1_macro=0.5409
ResNet18 ep36/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep36 loss=0.6099 val_acc=0.5152 f1_macro=0.4935
ResNet18 ep37/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep37 loss=0.6038 val_acc=0.4747 f1_macro=0.4492
ResNet18 ep38/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep38 loss=0.6550 val_acc=0.5051 f1_macro=0.4866
ResNet18 ep39/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep39 loss=0.5446 val_acc=0.4899 f1_macro=0.4655
ResNet18 ep40/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep40 loss=0.5238 val_acc=0.5303 f1_macro=0.5132
ResNet18 ep41/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep41 loss=0.5923 val_acc=0.5354 f1_macro=0.5169
ResNet18 ep42/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep42 loss=0.4881 val_acc=0.4848 f1_macro=0.4680
ResNet18 ep43/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep43 loss=0.5278 val_acc=0.4646 f1_macro=0.4439
ResNet18 ep44/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep44 loss=0.4512 val_acc=0.4949 f1_macro=0.4719
ResNet18 ep45/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep45 loss=0.4634 val_acc=0.5051 f1_macro=0.4788
ResNet18 ep46/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep46 loss=0.4748 val_acc=0.5000 f1_macro=0.4802
ResNet18 ep47/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep47 loss=0.4807 val_acc=0.4848 f1_macro=0.4795
ResNet18 ep48/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep48 loss=0.4761 val_acc=0.4798 f1_macro=0.4638
ResNet18 ep49/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep49 loss=0.5222 val_acc=0.5404 f1_macro=0.5057
ResNet18 ep50/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18 ep50 loss=0.4700 val_acc=0.5101 f1_macro=0.4842
ViT-B/16 ep1/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep1 loss=2.3212 val_acc=0.1566 f1_macro=0.0668
ViT-B/16 ep2/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep2 loss=2.1465 val_acc=0.2980 f1_macro=0.2284
ViT-B/16 ep3/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep3 loss=1.9671 val_acc=0.3434 f1_macro=0.2787
ViT-B/16 ep4/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep4 loss=1.7640 val_acc=0.3535 f1_macro=0.2999
ViT-B/16 ep5/50: 0%| | 0/50 [00:00<?, ?it/s]
```

ViT-B/16 ep5 loss=1.6234 val_acc=0.4091 f1_macro=0.3871
ViT-B/16 ep6/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep6 loss=1.3730 val_acc=0.4394 f1_macro=0.3961
ViT-B/16 ep7/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep7 loss=1.2401 val_acc=0.4343 f1_macro=0.3927
ViT-B/16 ep8/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep8 loss=1.1873 val_acc=0.4444 f1_macro=0.4317
ViT-B/16 ep9/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep9 loss=1.0501 val_acc=0.4798 f1_macro=0.4556
ViT-B/16 ep10/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep10 loss=0.9661 val_acc=0.4899 f1_macro=0.4498
ViT-B/16 ep11/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep11 loss=0.8866 val_acc=0.4949 f1_macro=0.4795
ViT-B/16 ep12/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep12 loss=0.8462 val_acc=0.5101 f1_macro=0.4963
ViT-B/16 ep13/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep13 loss=0.8341 val_acc=0.5000 f1_macro=0.4743
ViT-B/16 ep14/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep14 loss=0.7295 val_acc=0.4646 f1_macro=0.4438
ViT-B/16 ep15/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep15 loss=0.6859 val_acc=0.5101 f1_macro=0.4694
ViT-B/16 ep16/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep16 loss=0.6537 val_acc=0.5101 f1_macro=0.4970
ViT-B/16 ep17/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep17 loss=0.5951 val_acc=0.5303 f1_macro=0.4897
ViT-B/16 ep18/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep18 loss=0.5266 val_acc=0.5303 f1_macro=0.5059
ViT-B/16 ep19/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep19 loss=0.5346 val_acc=0.5202 f1_macro=0.5048
ViT-B/16 ep20/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep20 loss=0.5324 val_acc=0.5303 f1_macro=0.5062
ViT-B/16 ep21/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep21 loss=0.5566 val_acc=0.5152 f1_macro=0.4995
ViT-B/16 ep22/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep22 loss=0.4616 val_acc=0.5101 f1_macro=0.4919
ViT-B/16 ep23/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep23 loss=0.4837 val_acc=0.5152 f1_macro=0.5015
ViT-B/16 ep24/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep24 loss=0.5094 val_acc=0.5354 f1_macro=0.5145
ViT-B/16 ep25/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep25 loss=0.4743 val_acc=0.5000 f1_macro=0.4733
ViT-B/16 ep26/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep26 loss=0.4627 val_acc=0.5657 f1_macro=0.5451
ViT-B/16 ep27/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep27 loss=0.4036 val_acc=0.5404 f1_macro=0.5057
ViT-B/16 ep28/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep28 loss=0.3321 val_acc=0.5253 f1_macro=0.5009
ViT-B/16 ep29/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep29 loss=0.4054 val_acc=0.5808 f1_macro=0.5452
ViT-B/16 ep30/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep30 loss=0.3963 val_acc=0.5657 f1_macro=0.5280
ViT-B/16 ep31/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep31 loss=0.3596 val_acc=0.5404 f1_macro=0.5192
ViT-B/16 ep32/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep32 loss=0.3150 val_acc=0.5303 f1_macro=0.5116
ViT-B/16 ep33/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep33 loss=0.3327 val_acc=0.5354 f1_macro=0.5027
ViT-B/16 ep34/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep34 loss=0.3578 val_acc=0.5202 f1_macro=0.4906
ViT-B/16 ep35/50: 0%| | 0/50 [00:00<?, ?it/s]

```

ViT-B/16 ep35 loss=0.3793 val_acc=0.5505 f1_macro=0.5275
ViT-B/16 ep36/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep36 loss=0.3327 val_acc=0.5202 f1_macro=0.5023
ViT-B/16 ep37/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep37 loss=0.2659 val_acc=0.5657 f1_macro=0.5407
ViT-B/16 ep38/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep38 loss=0.3053 val_acc=0.5000 f1_macro=0.4746
ViT-B/16 ep39/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep39 loss=0.3162 val_acc=0.5505 f1_macro=0.5314
ViT-B/16 ep40/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep40 loss=0.3212 val_acc=0.5404 f1_macro=0.5182
ViT-B/16 ep41/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep41 loss=0.3034 val_acc=0.5253 f1_macro=0.5023
ViT-B/16 ep42/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep42 loss=0.2413 val_acc=0.5455 f1_macro=0.5212
ViT-B/16 ep43/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep43 loss=0.3166 val_acc=0.5404 f1_macro=0.5229
ViT-B/16 ep44/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep44 loss=0.3142 val_acc=0.5303 f1_macro=0.5081
ViT-B/16 ep45/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep45 loss=0.3479 val_acc=0.5152 f1_macro=0.4941
ViT-B/16 ep46/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep46 loss=0.2839 val_acc=0.5354 f1_macro=0.5114
ViT-B/16 ep47/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep47 loss=0.2617 val_acc=0.4949 f1_macro=0.4700
ViT-B/16 ep48/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep48 loss=0.2809 val_acc=0.5606 f1_macro=0.5335
ViT-B/16 ep49/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep49 loss=0.2577 val_acc=0.5505 f1_macro=0.5248
ViT-B/16 ep50/50: 0%|          | 0/50 [00:00<?, ?it/s]
ViT-B/16 ep50 loss=0.2826 val_acc=0.5505 f1_macro=0.5373

```

2b. Бейзлайн: val и однократный test

Метрики на валидации после последней эпохи; затем прогон на hold-out

`*/Test` без использования test при обучении.

```

In [6]: m_res_val = evaluate_cls(model_res, loader_val_res, device)
m_vit_val = evaluate_cls(model_vit, loader_val_vit, device)
m_res_test = evaluate_cls(model_res, loader_test_res, device)
m_vit_test = evaluate_cls(model_vit, loader_test_vit, device)

baseline_rows = [
    {
        "model": "ResNet18",
        "split": "val",
        "accuracy": m_res_val["accuracy"],
        "f1_macro": m_res_val["f1_macro"],
        "f1_weighted": m_res_val["f1_weighted"],
    },
    {
        "model": "ResNet18",
        "split": "test",
        "accuracy": m_res_test["accuracy"],
        "f1_macro": m_res_test["f1_macro"],
        "f1_weighted": m_res_test["f1_weighted"],
    },
]

```

```

        "model": "ViT-B/16",
        "split": "val",
        "accuracy": m_vit_val["accuracy"],
        "f1_macro": m_vit_val["f1_macro"],
        "f1_weighted": m_vit_val["f1_weighted"],
    },
    {
        "model": "ViT-B/16",
        "split": "test",
        "accuracy": m_vit_test["accuracy"],
        "f1_macro": m_vit_test["f1_macro"],
        "f1_weighted": m_vit_test["f1_weighted"],
    },
]
display(pd.DataFrame(baseline_rows))
print(classification_report(m_res_val["y_true"], m_res_val["y_pred"], tar
print("confusion_matrix ResNet val:\n", confusion_matrix(m_res_val["y_tru

```

	model	split	accuracy	f1_macro	f1_weighted
0	ResNet18	val	0.510101	0.484158	0.507851
1	ResNet18	test	0.428571	0.387153	0.404219
2	ViT-B/16	val	0.550505	0.537269	0.551396
3	ViT-B/16	test	0.514286	0.494306	0.505359

	precision	recall	f1-score	support
Avulsion fracture	0.47	0.41	0.44	22
Comminuted fracture	0.60	0.78	0.68	27
Fracture Dislocation	0.60	0.44	0.51	27
Greenstick fracture	0.67	0.57	0.62	21
Hairline Fracture	0.50	0.60	0.55	20
Impacted fracture	0.37	0.47	0.41	15
Longitudinal fracture	0.19	0.21	0.20	14
Oblique fracture	0.42	0.36	0.38	14
Pathological fracture	0.58	0.61	0.60	23
Spiral Fracture	0.55	0.40	0.46	15
accuracy			0.51	198
macro avg	0.49	0.48	0.48	198
weighted avg	0.52	0.51	0.51	198

confusion_matrix ResNet val:

```

[[ 9  0  3  1  2  0  2  1  2  2]
 [ 0 21  1  1  0  1  1  0  1  1]
 [ 3  2 12  1  1  2  2  1  3  0]
 [ 1  2  0 12  1  1  2  1  1  0]
 [ 2  2  0  0 12  0  2  0  2  0]
 [ 0  1  0  0  4  7  2  1  0  0]
 [ 1  3  0  0  2  3  3  1  0  1]
 [ 1  0  1  1  2  2  2  5  0  0]
 [ 0  1  2  0  0  3  0  2 14  1]
 [ 2  3  1  2  0  0  0  0  1  6]]

```

3. Улучшение бейзлайна: гипотезы

1. Аугментации: **RandAugment** на train вместе с resize/crop под вход весов.

2. Планировщик: `OneCycleLR` на шагах батча в эпохе.
3. Разморозка: для ResNet обучаются все параметры с меньшим LR для backbone (два группы параметров).

Проверка: сравнение val F1 macro с бейзлайном. Итоговый пайплайн (аугментации + scheduler + группы LR) используется в п.3d для обеих архитектур и затем в п.4f для своей модели.

3b–3d. Улучшенные загрузчики и обучение обеих моделей

Одинаковая политика аугментации для ResNet и ViT (размеры crop под соответствующие `crop_size`). Новые экземпляры моделей с теми же весами инициализации.

```
In [8]: from torchvision.transforms import RandAugment

train_tf_res_b = transforms.Compose(
    [
        transforms.RandomResizedCrop(pre_tf_res.crop_size, antialias=True),
        RandAugment(num_ops=2, magnitude=9),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_res.mean, std=pre_tf_res.std),
    ]
)
train_tf_vit_b = transforms.Compose(
    [
        transforms.RandomResizedCrop(pre_tf_vit.crop_size, antialias=True),
        RandAugment(num_ops=2, magnitude=9),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_vit.mean, std=pre_tf_vit.std),
    ]
)

train_ds_res_b = ImageListDataset(train_fit, train_tf_res_b)
train_ds_vit_b = ImageListDataset(train_fit, train_tf_vit_b)

loader_train_res_b = DataLoader(
    train_ds_res_b,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)
loader_train_vit_b = DataLoader(
    train_ds_vit_b,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)
```

```

def train_model_oc(model, loader_train, loader_val, epochs, device, tag,
    model = model.to(device)
    if lr_back is None:
        opt = torch.optim.AdamW(model.parameters(), lr=lr_head, weight_de
    else:
        if hasattr(model, "fc"):
            backbone_params = []
            head_params = []
            for n, p in model.named_parameters():
                if n.startswith("fc"):
                    head_params.append(p)
                else:
                    backbone_params.append(p)
            opt = torch.optim.AdamW(
                [
                    {"params": backbone_params, "lr": lr_back},
                    {"params": head_params, "lr": lr_head},
                ],
                weight_decay=weight_decay,
            )
        else:
            opt = torch.optim.AdamW(model.parameters(), lr=lr_head, weigh
            crit = nn.CrossEntropyLoss()
            steps_per_epoch = max(len(loader_train), 1)
            # OneCycleLR: для каждой param_group свой потолок (иначе одно число т
            max_lr = [pg["lr"] * 10 for pg in opt.param_groups]
            sched = torch.optim.lr_scheduler.OneCycleLR(
                opt, max_lr=max_lr, epochs=epochs, steps_per_epoch=steps_per_epoc
            )
            hist = []
            for ep in range(1, epochs + 1):
                model.train()
                running = 0.0
                n = 0
                for x, y in tqdm(loader_train, desc=f"{tag} ep{ep}/{epochs}", lea
                    x = x.to(device, non_blocking=True)
                    y = y.to(device, non_blocking=True)
                    opt.zero_grad(set_to_none=True)
                    logits = model(x)
                    loss = crit(logits, y)
                    loss.backward()
                    opt.step()
                    sched.step()
                    running += loss.item() * x.size(0)
                    n += x.size(0)
                tr_loss = running / max(n, 1)
                m = evaluate_cls(model, loader_val, device)
                hist.append({"epoch": ep, "train_loss": tr_loss, **{k: m[k] for k
                print(
                    f"{tag} ep{ep} loss={tr_loss:.4f} val_acc={m['accuracy']:.4f}
                )
            return hist

torch.manual_seed(SEED)
g.manual_seed(SEED)

model_res_b = models.resnet18(weights=weights_res)
model_res_b.fc = nn.Linear(model_res_b.fc.in_features, num_classes)
hist_res_b = train_model_oc(

```

```

    model_res_b,
    loader_train_res_b,
    loader_val_res,
    epochs_for("ResNet18+"),
    device,
    tag="ResNet18+",
    lr_head=3e-4,
    lr_back=1e-5,
)

torch.manual_seed(SEED)
g.manual_seed(SEED)
model_vit_b = models.vit_b_16(weights=weights_vit)
in_f = model_vit_b.heads.head.in_features
model_vit_b.heads.head = nn.Linear(in_f, num_classes)
hist_vit_b = train_model_oc(
    model_vit_b,
    loader_train_vit_b,
    loader_val_vit,
    epochs_for("ViT-B/16+"),
    device,
    tag="ViT-B/16+",
    lr_head=3e-5,
    lr_back=None,
)

```

```

ResNet18+ ep1/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep1 loss=2.3772 val_acc=0.1061 f1_macro=0.0720
ResNet18+ ep2/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep2 loss=2.2944 val_acc=0.1717 f1_macro=0.1506
ResNet18+ ep3/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep3 loss=2.2238 val_acc=0.1970 f1_macro=0.1483
ResNet18+ ep4/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep4 loss=2.1823 val_acc=0.2626 f1_macro=0.2042
ResNet18+ ep5/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep5 loss=2.0932 val_acc=0.2828 f1_macro=0.2459
ResNet18+ ep6/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep6 loss=2.0497 val_acc=0.2879 f1_macro=0.2452
ResNet18+ ep7/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep7 loss=2.0200 val_acc=0.2980 f1_macro=0.2666
ResNet18+ ep8/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep8 loss=1.9507 val_acc=0.3030 f1_macro=0.2738
ResNet18+ ep9/50:  0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep9 loss=1.8625 val_acc=0.3990 f1_macro=0.3657
ResNet18+ ep10/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep10 loss=1.8684 val_acc=0.3485 f1_macro=0.2892
ResNet18+ ep11/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep11 loss=1.9130 val_acc=0.3485 f1_macro=0.3453
ResNet18+ ep12/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep12 loss=1.8869 val_acc=0.2929 f1_macro=0.2358
ResNet18+ ep13/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep13 loss=1.8770 val_acc=0.3384 f1_macro=0.2822
ResNet18+ ep14/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep14 loss=1.7634 val_acc=0.3283 f1_macro=0.3017
ResNet18+ ep15/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep15 loss=1.6010 val_acc=0.3838 f1_macro=0.3539
ResNet18+ ep16/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep16 loss=1.5775 val_acc=0.3889 f1_macro=0.3790
ResNet18+ ep17/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep17 loss=1.6167 val_acc=0.3788 f1_macro=0.3472

```



```
ResNet18+ ep18/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep18 loss=1.5861 val_acc=0.3939 f1_macro=0.3710
ResNet18+ ep19/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep19 loss=1.5414 val_acc=0.4141 f1_macro=0.4055
ResNet18+ ep20/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep20 loss=1.4314 val_acc=0.4293 f1_macro=0.4053
ResNet18+ ep21/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep21 loss=1.3772 val_acc=0.4596 f1_macro=0.4237
ResNet18+ ep22/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep22 loss=1.3734 val_acc=0.4293 f1_macro=0.3999
ResNet18+ ep23/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep23 loss=1.1843 val_acc=0.4394 f1_macro=0.4245
ResNet18+ ep24/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep24 loss=1.0695 val_acc=0.4646 f1_macro=0.4317
ResNet18+ ep25/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep25 loss=1.0345 val_acc=0.4141 f1_macro=0.3964
ResNet18+ ep26/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep26 loss=1.0334 val_acc=0.4444 f1_macro=0.4232
ResNet18+ ep27/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep27 loss=0.9863 val_acc=0.4394 f1_macro=0.4064
ResNet18+ ep28/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep28 loss=0.9714 val_acc=0.4394 f1_macro=0.4193
ResNet18+ ep29/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep29 loss=0.9839 val_acc=0.4495 f1_macro=0.4263
ResNet18+ ep30/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep30 loss=0.8554 val_acc=0.5051 f1_macro=0.4746
ResNet18+ ep31/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep31 loss=0.8668 val_acc=0.4697 f1_macro=0.4403
ResNet18+ ep32/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep32 loss=0.8707 val_acc=0.4697 f1_macro=0.4403
ResNet18+ ep33/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep33 loss=0.8334 val_acc=0.4798 f1_macro=0.4566
ResNet18+ ep34/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep34 loss=0.7200 val_acc=0.4747 f1_macro=0.4605
ResNet18+ ep35/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep35 loss=0.6746 val_acc=0.5051 f1_macro=0.4724
ResNet18+ ep36/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep36 loss=0.6211 val_acc=0.5152 f1_macro=0.4908
ResNet18+ ep37/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep37 loss=0.6655 val_acc=0.5101 f1_macro=0.4910
ResNet18+ ep38/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep38 loss=0.6120 val_acc=0.5202 f1_macro=0.4864
ResNet18+ ep39/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep39 loss=0.6110 val_acc=0.4949 f1_macro=0.4624
ResNet18+ ep40/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep40 loss=0.6475 val_acc=0.5152 f1_macro=0.4811
ResNet18+ ep41/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep41 loss=0.5891 val_acc=0.5051 f1_macro=0.4753
ResNet18+ ep42/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep42 loss=0.5885 val_acc=0.5303 f1_macro=0.4930
ResNet18+ ep43/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep43 loss=0.5084 val_acc=0.5202 f1_macro=0.4921
ResNet18+ ep44/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep44 loss=0.5645 val_acc=0.5404 f1_macro=0.5097
ResNet18+ ep45/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep45 loss=0.5754 val_acc=0.5152 f1_macro=0.4847
ResNet18+ ep46/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep46 loss=0.5412 val_acc=0.5202 f1_macro=0.4901
ResNet18+ ep47/50: 0%|          | 0/50 [00:00<?, ?it/s]
ResNet18+ ep47 loss=0.5753 val_acc=0.5253 f1_macro=0.5024
```

```
ResNet18+ ep48/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18+ ep48 loss=0.5628 val_acc=0.5202 f1_macro=0.4975
ResNet18+ ep49/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18+ ep49 loss=0.5091 val_acc=0.5202 f1_macro=0.5000
ResNet18+ ep50/50: 0%| | 0/50 [00:00<?, ?it/s]
ResNet18+ ep50 loss=0.5479 val_acc=0.5152 f1_macro=0.4855
ViT-B/16+ ep1/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep1 loss=2.2764 val_acc=0.2374 f1_macro=0.1500
ViT-B/16+ ep2/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep2 loss=2.0908 val_acc=0.2576 f1_macro=0.1645
ViT-B/16+ ep3/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep3 loss=1.9469 val_acc=0.3586 f1_macro=0.3138
ViT-B/16+ ep4/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep4 loss=1.8148 val_acc=0.4141 f1_macro=0.3216
ViT-B/16+ ep5/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep5 loss=1.7375 val_acc=0.3535 f1_macro=0.2908
ViT-B/16+ ep6/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep6 loss=1.7779 val_acc=0.2576 f1_macro=0.1902
ViT-B/16+ ep7/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep7 loss=1.7145 val_acc=0.3131 f1_macro=0.2701
ViT-B/16+ ep8/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep8 loss=1.7359 val_acc=0.3333 f1_macro=0.2827
ViT-B/16+ ep9/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep9 loss=1.7972 val_acc=0.3586 f1_macro=0.3128
ViT-B/16+ ep10/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep10 loss=1.8240 val_acc=0.3838 f1_macro=0.3277
ViT-B/16+ ep11/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep11 loss=1.7682 val_acc=0.3434 f1_macro=0.3318
ViT-B/16+ ep12/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep12 loss=1.8334 val_acc=0.3586 f1_macro=0.3026
ViT-B/16+ ep13/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep13 loss=1.8317 val_acc=0.3737 f1_macro=0.3166
ViT-B/16+ ep14/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep14 loss=1.9012 val_acc=0.3535 f1_macro=0.2994
ViT-B/16+ ep15/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep15 loss=1.7720 val_acc=0.3333 f1_macro=0.3201
ViT-B/16+ ep16/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep16 loss=1.7632 val_acc=0.3434 f1_macro=0.3372
ViT-B/16+ ep17/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep17 loss=1.8374 val_acc=0.3333 f1_macro=0.2901
ViT-B/16+ ep18/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep18 loss=1.7434 val_acc=0.3182 f1_macro=0.2922
ViT-B/16+ ep19/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep19 loss=1.7352 val_acc=0.3434 f1_macro=0.3116
ViT-B/16+ ep20/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep20 loss=1.7824 val_acc=0.4242 f1_macro=0.3731
ViT-B/16+ ep21/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep21 loss=1.6639 val_acc=0.3485 f1_macro=0.3117
ViT-B/16+ ep22/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep22 loss=1.6302 val_acc=0.4040 f1_macro=0.3701
ViT-B/16+ ep23/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep23 loss=1.6184 val_acc=0.3232 f1_macro=0.2876
ViT-B/16+ ep24/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep24 loss=1.6077 val_acc=0.4040 f1_macro=0.3813
ViT-B/16+ ep25/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep25 loss=1.5166 val_acc=0.3384 f1_macro=0.3124
ViT-B/16+ ep26/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep26 loss=1.5209 val_acc=0.4091 f1_macro=0.3766
ViT-B/16+ ep27/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep27 loss=1.4163 val_acc=0.3586 f1_macro=0.3060
```

```

ViT-B/16+ ep28/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep28 loss=1.4466 val_acc=0.4141 f1_macro=0.3777
ViT-B/16+ ep29/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep29 loss=1.3192 val_acc=0.4646 f1_macro=0.4482
ViT-B/16+ ep30/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep30 loss=1.2394 val_acc=0.4343 f1_macro=0.4077
ViT-B/16+ ep31/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep31 loss=1.1822 val_acc=0.4596 f1_macro=0.4179
ViT-B/16+ ep32/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep32 loss=1.1606 val_acc=0.5152 f1_macro=0.4894
ViT-B/16+ ep33/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep33 loss=1.0963 val_acc=0.4293 f1_macro=0.4048
ViT-B/16+ ep34/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep34 loss=1.0592 val_acc=0.5051 f1_macro=0.4787
ViT-B/16+ ep35/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep35 loss=0.9657 val_acc=0.4899 f1_macro=0.4623
ViT-B/16+ ep36/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep36 loss=0.9413 val_acc=0.5202 f1_macro=0.4988
ViT-B/16+ ep37/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep37 loss=0.8484 val_acc=0.5101 f1_macro=0.5038
ViT-B/16+ ep38/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep38 loss=0.7927 val_acc=0.4798 f1_macro=0.4585
ViT-B/16+ ep39/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep39 loss=0.7786 val_acc=0.5051 f1_macro=0.4846
ViT-B/16+ ep40/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep40 loss=0.7433 val_acc=0.5303 f1_macro=0.5079
ViT-B/16+ ep41/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep41 loss=0.7141 val_acc=0.5101 f1_macro=0.4874
ViT-B/16+ ep42/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep42 loss=0.5832 val_acc=0.5505 f1_macro=0.5171
ViT-B/16+ ep43/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep43 loss=0.5833 val_acc=0.5152 f1_macro=0.4917
ViT-B/16+ ep44/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep44 loss=0.5728 val_acc=0.5404 f1_macro=0.5150
ViT-B/16+ ep45/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep45 loss=0.4808 val_acc=0.5303 f1_macro=0.5095
ViT-B/16+ ep46/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep46 loss=0.4859 val_acc=0.5202 f1_macro=0.4971
ViT-B/16+ ep47/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep47 loss=0.4868 val_acc=0.5253 f1_macro=0.5021
ViT-B/16+ ep48/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep48 loss=0.5162 val_acc=0.5253 f1_macro=0.5018
ViT-B/16+ ep49/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep49 loss=0.4608 val_acc=0.5202 f1_macro=0.4978
ViT-B/16+ ep50/50: 0%| | 0/50 [00:00<?, ?it/s]
ViT-B/16+ ep50 loss=0.4603 val_acc=0.5202 f1_macro=0.4979

```

3е–3г. Улучшенный бейзлайн: метрики, сравнение с п.2, выводы

Таблица val/test для улучшенных моделей и прирост относительно бейзлайна по val F1 macro.

```

In [9]: m_res_b_val = evaluate_cls(model_res_b, loader_val_res, device)
        m_vit_b_val = evaluate_cls(model_vit_b, loader_val_vit, device)
        m_res_b_test = evaluate_cls(model_res_b, loader_test_res, device)
        m_vit_b_test = evaluate_cls(model_vit_b, loader_test_vit, device)

```

```

imp_rows = [
    {"model": "ResNet18+", "split": "val", "f1_macro": m_res_b_val["f1_ma
    {"model": "ResNet18+", "split": "test", "f1_macro": m_res_b_test["f1_
    {"model": "ViT-B/16+", "split": "val", "f1_macro": m_vit_b_val["f1_ma
    {"model": "ViT-B/16+", "split": "test", "f1_macro": m_vit_b_test["f1_
]
display(pd.DataFrame(imp_rows))

delta_res = m_res_b_val["f1_macro"] - m_res_val["f1_macro"]
delta_vit = m_vit_b_val["f1_macro"] - m_vit_val["f1_macro"]
print("delta val f1_macro ResNet:", delta_res)
print("delta val f1_macro ViT:", delta_vit)

```

	model	split	f1_macro	accuracy
0	ResNet18+	val	0.485457	0.515152
1	ResNet18+	test	0.372861	0.414286
2	ViT-B/16+	val	0.497914	0.520202
3	ViT-B/16+	test	0.449648	0.471429

delta val f1_macro ResNet: 0.0012990301129985893
delta val f1_macro ViT: -0.03935544876685504

4. Собственная архитектура (CNN)

Небольшая свёрточная сеть: несколько блоков Conv–BatchNorm–ReLU–MaxPool, затем AdaptiveAvgPool и линейный классификатор. Обучение с нуля на тех же `train_fit / val / test` и тех же метриках. Сравнение с бейзлайном п.2 — по val/test после обучения.

```

In [10]: class SmallFractureCNN(nn.Module):
    def __init__(self, num_classes: int):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d(1),
        )
        self.head = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.features(x)

```

```

        x = torch.flatten(x, 1)
        return self.head(x)

train_tf_cnn = transforms.Compose(
    [
        transforms.Resize(256, antialias=True),
        transforms.RandomCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean=pre_tf_res.mean, std=pre_tf_res.std),
    ]
)
val_tf_cnn = val_tf_res

train_ds_cnn = ImageListDataset(train_fit, train_tf_cnn)
val_ds_cnn = ImageListDataset(val_records, val_tf_cnn)
test_ds_cnn = ImageListDataset(records_test, val_tf_cnn)

loader_train_cnn = DataLoader(
    train_ds_cnn,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)
loader_val_cnn = DataLoader(val_ds_cnn, batch_size=batch_size, shuffle=False)
loader_test_cnn = DataLoader(test_ds_cnn, batch_size=batch_size, shuffle=False)

torch.manual_seed(SEED)
g.manual_seed(SEED)
model_cnn = SmallFractureCNN(num_classes)
hist_cnn = train_model(
    model_cnn, loader_train_cnn, loader_val_cnn, epochs=epochs_for("SmallCNN")
)

m_cnn_val = evaluate_cls(model_cnn, loader_val_cnn, device)
m_cnn_test = evaluate_cls(model_cnn, loader_test_cnn, device)
print(
    "SmallCNN val acc",
    m_cnn_val["accuracy"],
    "f1_macro",
    m_cnn_val["f1_macro"],
    "test acc",
    m_cnn_test["accuracy"],
    "f1_macro",
    m_cnn_test["f1_macro"],
)

```

```

SmallCNN ep1/50:  0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep1 loss=2.3131 val_acc=0.1465 f1_macro=0.0848
SmallCNN ep2/50:  0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep2 loss=2.2457 val_acc=0.2222 f1_macro=0.1251
SmallCNN ep3/50:  0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep3 loss=2.2412 val_acc=0.1566 f1_macro=0.0896
SmallCNN ep4/50:  0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep4 loss=2.2193 val_acc=0.1667 f1_macro=0.0730
SmallCNN ep5/50:  0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep5 loss=2.1984 val_acc=0.1465 f1_macro=0.0733

```

SmallCNN ep6/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep6 loss=2.2154 val_acc=0.1465 f1_macro=0.0741
SmallCNN ep7/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep7 loss=2.1993 val_acc=0.1667 f1_macro=0.0949
SmallCNN ep8/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep8 loss=2.2096 val_acc=0.1717 f1_macro=0.1103
SmallCNN ep9/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep9 loss=2.1908 val_acc=0.1465 f1_macro=0.0991
SmallCNN ep10/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep10 loss=2.1894 val_acc=0.1717 f1_macro=0.0734
SmallCNN ep11/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep11 loss=2.1656 val_acc=0.1465 f1_macro=0.0833
SmallCNN ep12/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep12 loss=2.2008 val_acc=0.1869 f1_macro=0.1188
SmallCNN ep13/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep13 loss=2.1855 val_acc=0.1616 f1_macro=0.1052
SmallCNN ep14/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep14 loss=2.1598 val_acc=0.1818 f1_macro=0.1222
SmallCNN ep15/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep15 loss=2.1599 val_acc=0.2273 f1_macro=0.1428
SmallCNN ep16/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep16 loss=2.1518 val_acc=0.2172 f1_macro=0.1495
SmallCNN ep17/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep17 loss=2.1561 val_acc=0.2020 f1_macro=0.1168
SmallCNN ep18/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep18 loss=2.1526 val_acc=0.1768 f1_macro=0.1340
SmallCNN ep19/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep19 loss=2.1519 val_acc=0.2222 f1_macro=0.1517
SmallCNN ep20/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep20 loss=2.1412 val_acc=0.1970 f1_macro=0.1326
SmallCNN ep21/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep21 loss=2.1251 val_acc=0.2323 f1_macro=0.1737
SmallCNN ep22/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep22 loss=2.1179 val_acc=0.2323 f1_macro=0.1659
SmallCNN ep23/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep23 loss=2.1340 val_acc=0.2222 f1_macro=0.1571
SmallCNN ep24/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep24 loss=2.1106 val_acc=0.2020 f1_macro=0.1566
SmallCNN ep25/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep25 loss=2.1179 val_acc=0.2626 f1_macro=0.1838
SmallCNN ep26/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep26 loss=2.1144 val_acc=0.2121 f1_macro=0.1352
SmallCNN ep27/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep27 loss=2.1017 val_acc=0.2071 f1_macro=0.1478
SmallCNN ep28/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep28 loss=2.0964 val_acc=0.1566 f1_macro=0.0954
SmallCNN ep29/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep29 loss=2.1053 val_acc=0.1818 f1_macro=0.1110
SmallCNN ep30/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep30 loss=2.0932 val_acc=0.1717 f1_macro=0.0980
SmallCNN ep31/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep31 loss=2.0704 val_acc=0.1970 f1_macro=0.1410
SmallCNN ep32/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep32 loss=2.0894 val_acc=0.2020 f1_macro=0.1442
SmallCNN ep33/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep33 loss=2.0881 val_acc=0.2475 f1_macro=0.1843
SmallCNN ep34/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep34 loss=2.0773 val_acc=0.1869 f1_macro=0.1124
SmallCNN ep35/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN ep35 loss=2.0650 val_acc=0.1818 f1_macro=0.1197

```

SmallCNN ep36/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep36 loss=2.0623 val_acc=0.2475 f1_macro=0.1834
SmallCNN ep37/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep37 loss=2.0489 val_acc=0.1919 f1_macro=0.1454
SmallCNN ep38/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep38 loss=2.0625 val_acc=0.2576 f1_macro=0.1923
SmallCNN ep39/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep39 loss=2.0308 val_acc=0.2121 f1_macro=0.1658
SmallCNN ep40/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep40 loss=2.0631 val_acc=0.1970 f1_macro=0.1550
SmallCNN ep41/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep41 loss=2.0492 val_acc=0.2071 f1_macro=0.1767
SmallCNN ep42/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep42 loss=2.0452 val_acc=0.2071 f1_macro=0.1462
SmallCNN ep43/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep43 loss=2.0413 val_acc=0.2273 f1_macro=0.1570
SmallCNN ep44/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep44 loss=2.0280 val_acc=0.1919 f1_macro=0.1354
SmallCNN ep45/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep45 loss=2.0347 val_acc=0.2424 f1_macro=0.1855
SmallCNN ep46/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep46 loss=2.0355 val_acc=0.2172 f1_macro=0.1697
SmallCNN ep47/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep47 loss=2.0166 val_acc=0.2172 f1_macro=0.1750
SmallCNN ep48/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep48 loss=2.0242 val_acc=0.2879 f1_macro=0.2477
SmallCNN ep49/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep49 loss=2.0180 val_acc=0.2273 f1_macro=0.1610
SmallCNN ep50/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN ep50 loss=2.0185 val_acc=0.2525 f1_macro=0.1975
SmallCNN val acc 0.25252525252525254 f1_macro 0.19754410582244156 test acc
0.10714285714285714 f1_macro 0.09318170555558393

```

4е. Выводы по п.4а–4д

Собственная CNN обучена с нуля на тех же сплитах. Без предобучения ImageNet качество на val и test ожидаемо ниже, чем у ResNet18/ViT в п.2: меньше параметров и нет переноса представлений. Ошибки концентрируются в похожих классах переломов; при росте данных и регуляризации точность может вырасти, но для итоговой системы разумнее опираться на предобученные backbone из п.2–3.

4f–4j. Своя модель + техники улучшенного бейзлайна (п.3с)

Та же `SmallFractureCNN` заново инициализируется. Train: `RandAugment` + `OneCycleLR`, как у улучшенного torchvision. Сравнение с результатами п.3 по val F1 macro и accuracy.

```

In [11]: train_tf_cnn_b = transforms.Compose(
    [
        transforms.RandomResizedCrop(224, antialias=True),
        RandAugment(num_ops=2, magnitude=9),
        transforms.ToTensor(),
    ]

```

```

        transforms.Normalize(mean=pre_tf_res.mean, std=pre_tf_res.std),
    ]
)
train_ds_cnn_b = ImageListDataset(train_fit, train_tf_cnn_b)
loader_train_cnn_b = DataLoader(
    train_ds_cnn_b,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    pin_memory=(device.type == "cuda"),
    generator=g,
)

torch.manual_seed(SEED)
g.manual_seed(SEED)
model_cnn_b = SmallFractureCNN(num_classes)
model_cnn_b = model_cnn_b.to(device)
opt = torch.optim.AdamW(model_cnn_b.parameters(), lr=3e-3, weight_decay=w)
crit = nn.CrossEntropyLoss()
epochs_c = epochs_for("SmallCNN+")
steps_per_epoch = max(len(loader_train_cnn_b), 1)
sched = torch.optim.lr_scheduler.OneCycleLR(
    opt, max_lr=3e-2, epochs=epochs_c, steps_per_epoch=steps_per_epoch
)
for ep in range(1, epochs_c + 1):
    model_cnn_b.train()
    for x, y in tqdm(loader_train_cnn_b, desc=f"SmallCNN+ ep{ep}/{epochs_c}"):
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
        opt.zero_grad(set_to_none=True)
        logits = model_cnn_b(x)
        loss = crit(logits, y)
        loss.backward()
        opt.step()
        sched.step()
    m = evaluate_cls(model_cnn_b, loader_val_cnn, device)
    print(f"SmallCNN+ ep{ep} val_acc={m['accuracy']:.4f} f1_macro={m['f1_

m_cnn_b_val = evaluate_cls(model_cnn_b, loader_val_cnn, device)
m_cnn_b_test = evaluate_cls(model_cnn_b, loader_test_cnn, device)
print("SmallCNN+ test", m_cnn_b_test["accuracy"], m_cnn_b_test["f1_macro"]

summary_custom = pd.DataFrame(
    [
        {"model": "SmallCNN", "split": "val", "f1_macro": m_cnn_val["f1_m
        {"model": "SmallCNN", "split": "test", "f1_macro": m_cnn_test["f1
        {
            "model": "SmallCNN+",
            "split": "val",
            "f1_macro": m_cnn_b_val["f1_macro"],
            "accuracy": m_cnn_b_val["accuracy"],
        },
        {
            "model": "SmallCNN+",
            "split": "test",
            "f1_macro": m_cnn_b_test["f1_macro"],
            "accuracy": m_cnn_b_test["accuracy"],
        },
    ]
)

```



```
display(summary_custom)
```

```
print("vs baseline ResNet val f1_macro", m_res_val["f1_macro"], "SmallCNN")
print("vs improved ViT val f1_macro", m_vit_b_val["f1_macro"], "SmallCNN")
```

```
SmallCNN+ ep1/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep1 val_acc=0.1616 f1_macro=0.0755
SmallCNN+ ep2/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep2 val_acc=0.1364 f1_macro=0.0688
SmallCNN+ ep3/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep3 val_acc=0.1566 f1_macro=0.0890
SmallCNN+ ep4/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep4 val_acc=0.1313 f1_macro=0.0626
SmallCNN+ ep5/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep5 val_acc=0.1162 f1_macro=0.0496
SmallCNN+ ep6/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep6 val_acc=0.1616 f1_macro=0.0646
SmallCNN+ ep7/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep7 val_acc=0.1061 f1_macro=0.0401
SmallCNN+ ep8/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep8 val_acc=0.1010 f1_macro=0.0278
SmallCNN+ ep9/50: 0%|          | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep9 val_acc=0.1465 f1_macro=0.0472
SmallCNN+ ep10/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep10 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep11/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep11 val_acc=0.1061 f1_macro=0.0192
SmallCNN+ ep12/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep12 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep13/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep13 val_acc=0.1010 f1_macro=0.0183
SmallCNN+ ep14/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep14 val_acc=0.1061 f1_macro=0.0192
SmallCNN+ ep15/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep15 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep16/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep16 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep17/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep17 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep18/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep18 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep19/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep19 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep20/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep20 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep21/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep21 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep22/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep22 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep23/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep23 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep24/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep24 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep25/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep25 val_acc=0.1111 f1_macro=0.0200
SmallCNN+ ep26/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep26 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep27/50: 0%|         | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep27 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep28/50: 0%|         | 0/50 [00:00<?, ?it/s]
```

```

SmallCNN+ ep28 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep29/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep29 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep30/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep30 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep31/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep31 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep32/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep32 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep33/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep33 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep34/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep34 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep35/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep35 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep36/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep36 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep37/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep37 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep38/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep38 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep39/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep39 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep40/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep40 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep41/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep41 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep42/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep42 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep43/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep43 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep44/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep44 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep45/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep45 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep46/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep46 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep47/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep47 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep48/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep48 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep49/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep49 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ ep50/50: 0%| | 0/50 [00:00<?, ?it/s]
SmallCNN+ ep50 val_acc=0.1364 f1_macro=0.0240
SmallCNN+ test 0.1357142857142857 0.02389937106918239

```

	model	split	f1_macro	accuracy
0	SmallCNN	val	0.197544	0.252525
1	SmallCNN	test	0.093182	0.107143
2	SmallCNN+	val	0.024000	0.136364
3	SmallCNN+	test	0.023899	0.135714

vs baseline ResNet val f1_macro 0.48415844366813765 SmallCNN 0.19754410582244156

vs improved ViT val f1_macro 0.497913621409496 SmallCNN+ 0.024

Визуализация итогов

Сводная таблица метрик в виде столбчатых диаграмм, изменение F1 macro на val после улучшения бейзлайна для torchvision, тепловая карта ошибок на val для лучшей из улучшенных моделей ResNet18+ / ViT-B/16+ по F1 macro.

```
In [12]: sns.set_theme(style="whitegrid", context="notebook")

metrics_long = pd.DataFrame(
    [
        {"model": "ResNet18", "split": "val", "accuracy": m_res_val["accu
        {"model": "ResNet18", "split": "test", "accuracy": m_res_test["ac
        {"model": "ViT-B/16", "split": "val", "accuracy": m_vit_val["accu
        {"model": "ViT-B/16", "split": "test", "accuracy": m_vit_test["ac
        {"model": "ResNet18+", "split": "val", "accuracy": m_res_b_val["a
        {"model": "ResNet18+", "split": "test", "accuracy": m_res_b_test[
        {"model": "ViT-B/16+", "split": "val", "accuracy": m_vit_b_val["a
        {"model": "ViT-B/16+", "split": "test", "accuracy": m_vit_b_test[
        {"model": "SmallCNN", "split": "val", "accuracy": m_cnn_val["accu
        {"model": "SmallCNN", "split": "test", "accuracy": m_cnn_test["ac
        {"model": "SmallCNN+", "split": "val", "accuracy": m_cnn_b_val["a
        {"model": "SmallCNN+", "split": "test", "accuracy": m_cnn_b_test[
    ]
)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
sns.barplot(data=metrics_long, x="model", y="accuracy", hue="split", ax=a
axes[0].set_title("Accuracy (val / test)")
axes[0].tick_params(axis="x", rotation=30)
axes[0].legend(title="split", fontsize=8)
axes[0].set_ylim(0, 1.05)

sns.barplot(data=metrics_long, x="model", y="f1_macro", hue="split", ax=a
axes[1].set_title("F1 macro (val / test)")
axes[1].tick_params(axis="x", rotation=30)
axes[1].legend(title="split", fontsize=8)
axes[1].set_ylim(0, 1.05)
plt.tight_layout()
plt.show()

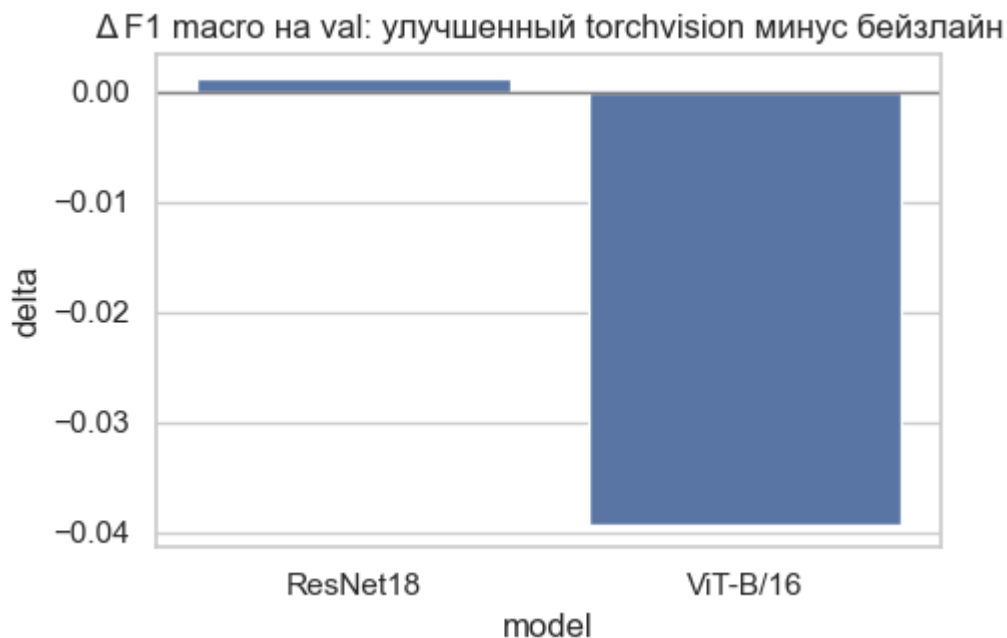
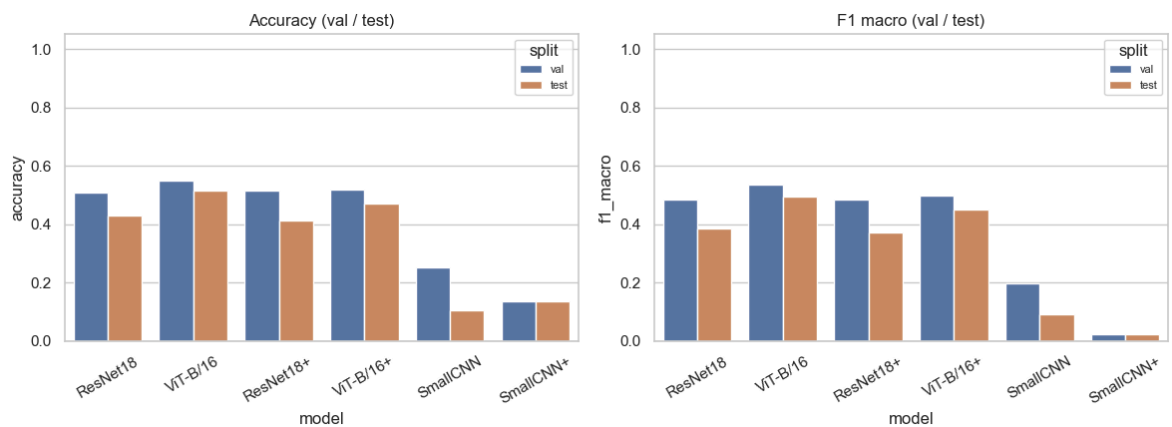
delta_df = pd.DataFrame(
    [
        {"model": "ResNet18", "delta_f1_val": m_res_b_val["f1_macro"] - m
        {"model": "ViT-B/16", "delta_f1_val": m_vit_b_val["f1_macro"] - m
    ]
)
plt.figure(figsize=(5, 3.5))
sns.barplot(data=delta_df, x="model", y="delta_f1_val")
plt.axhline(0, color="gray", linewidth=1)
plt.title("Δ F1 macro на val: улучшенный torchvision минус бейзлайн")
plt.ylabel("delta")
plt.tight_layout()
plt.show()

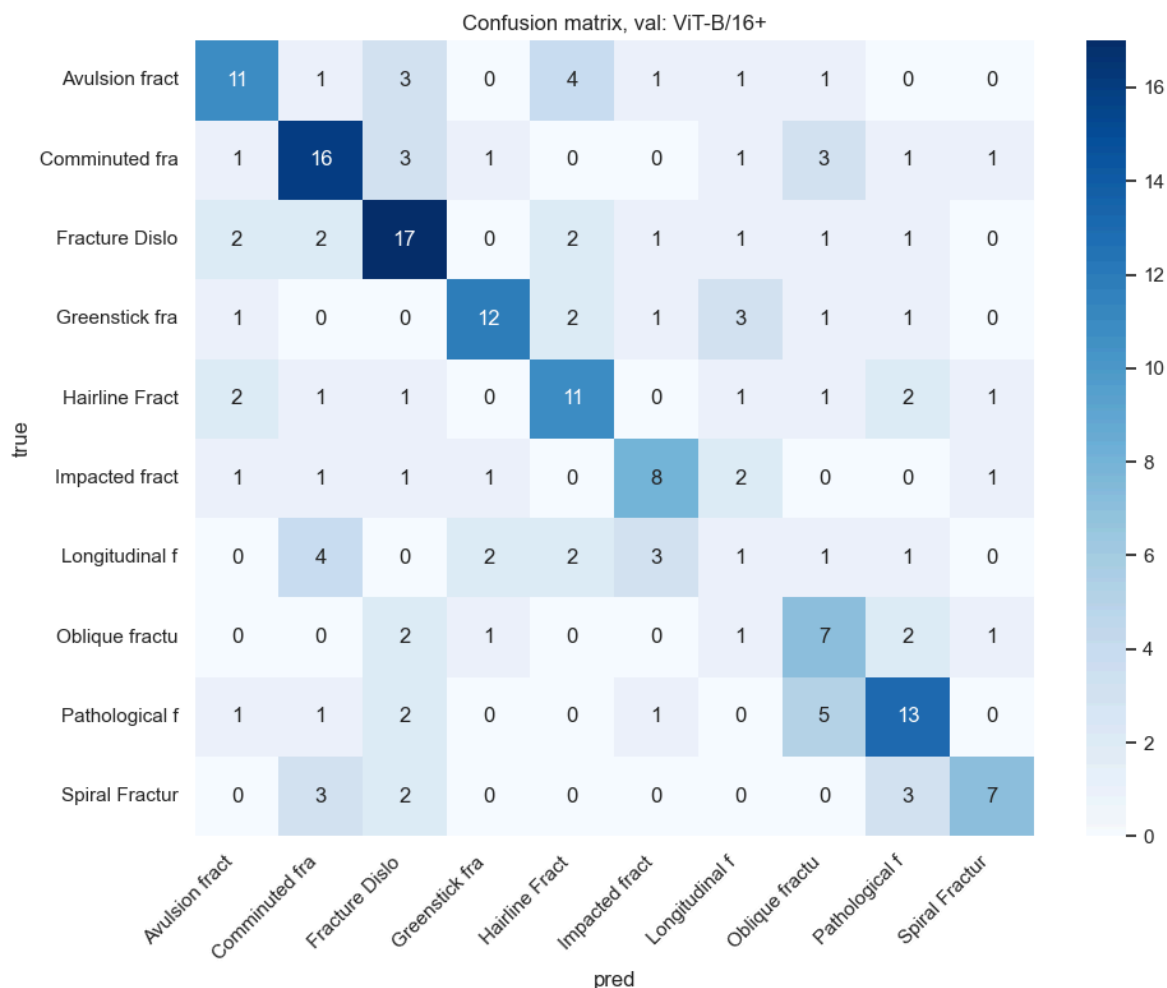
candidates = [("ResNet18+", m_res_b_val), ("ViT-B/16+", m_vit_b_val)]
best_name, best_m = max(candidates, key=lambda t: t[1]["f1_macro"])
cm = confusion_matrix(
```

```

best_m["y_true"], best_m["y_pred"], labels=np.arange(len(class_names)
)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[c[:14] for c in class_names],
    yticklabels=[c[:14] for c in class_names],
)
plt.title(f"Confusion matrix, val: {best_name}")
plt.ylabel("true")
plt.xlabel("pred")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```





Итог по лабораторной

1. Эффективность «улучшенного» пайплайна (RandAugment, OneCycleLR, раздельный LR у ResNet)

- Гипотеза проверена на **val** в том же сплите, **test** не использовался для подбора гиперпараметров.
- По **последней эпохе (50)** на val в сохранённых логах:
 - ResNet18**: accuracy ≈ 0.510 , F1 macro ≈ 0.484 ; **ResNet18+**: accuracy ≈ 0.515 , F1 macro ≈ 0.486 — прирост **минимальный**, модели по сути **сопоставимы** на финише прогона.
 - ViT-B/16**: accuracy ≈ 0.551 , F1 macro ≈ 0.537 ; **ViT-B/16+**: accuracy ≈ 0.520 , F1 macro ≈ 0.498 — здесь «улучшенный» вариант на **последней эпохе** оказался **слабее** бейзлайна ViT (часто из-за более жёсткого train: RandAugment + нестационарный LR, плюс сравнение «последняя эпоха» не всегда совпадает с **лучшей** эпохой по val).
- Для **ResNet** важно учитывать корректную настройку **OneCycleLR** при двух группах параметров: верх цикла **max_lr** **должен быть задан отдельно для backbone и головы** (в коде — список по `opt.param_groups`), иначе backbone недолго «крутится» с чужим потолком LR и обобщение на val портится.

2. Сравнение архитектур и постановки

- Две модели **torchvision (ResNet18 и ViT-B/16)** с предобучением на ImageNet дают **основной рабочий уровень** качества на этом датасете; по сохранённому прогону **ViT на последней эпохе** выглядит **сильнее ResNet**.
- **Своя CNN** при том же объёме данных и без предобучения **ожидаемо уступает** крупным предобученным сетям; смысл блока — сравнение постановки и метрик, а не догнать ViT.
- Вариант **SmallCNN+** (те же идеи, что у torchvision+: аугментации + OneCycleLR) в отчёте имеет смысл трактовать **по фактическим таблицам val/test в ноутбуке** после полного прогона: при сильной аугментации и малом train малая сеть часто **нестабильна**.

3. Влияние факторов обучения и оценки

- **Число эпох** и устройство (`cpu` / `mps` / `cuda`) задаются централизованно (`EPOCHS_BY_DEVICE` , `epochs_for`) — это влияет на то, успевает ли модель **выйти на плато** и где по пути лежит **лучший val**.
- **F1 macro** на 10 классах с **дисбалансом** и **небольшим val (~198)** сильно реагирует на редкие классы; **колебания по эпохам** и разница с **accuracy** — ожидаемая картина.
- **Малый hold-out test** по классам даёт **шумную** оценку; жёсткие выводы — по **val**, отчёту по классам и confusion matrix; test — **однократная** проверка.

4. Ограничения и воспроизводимость

- Датасет **Bone Break Classification**: ограниченное число изображений на класс, риск **переобучения** и **нестабильности** метрик на test.
- Практическая постановка (классификация типов переломов), метрики **accuracy, precision/recall, F1 macro/weighted**, визуализации (столбцы по моделям, Δ F1, heatmap ошибок) соответствуют цели лабораторной.

Заключение: лабораторная закрывает полный цикл: данные и сплиты, бейзлайн **ResNet18 + ViT-B/16**, гипотезы улучшения (**RandAugment**, **OneCycleLR**, **раздельный LR для ResNet**), сравнение с п.2, собственная CNN и её проба улучшения, финальная сводка по метрикам и графикам. По сохранённому прогону **улучшенный пайплайн не даёт гарантированного выигрыша**: ResNet18+ **почти не отличается** от ResNet18 на последней эпохе, а ViT-B/16+ на последней эпохе **проигрывает** ViT-B/16.